The numeral function is primitive recursive

The Open Logic Project contributors

openlogic:incompleteness/arithmetization-syntax:local-snapshot-20261013; CC BY 4.0

Editorial note. The counted statement is the author proposition labelled prop:num-primrec (733-736); the proof span 738-747 is the author proof environment that immediately follows it.

Editorial note. No cross-references occur in the retained span.

Editorial scope. One original result of the Open Logic Project unit incompleteness/arithmetization-syntax, counted once: the numeral function is primitive recursive. Only the counted proposition and its own author proof are retained; the source is reproduced without mathematical changes and keeps the CC BY 4.0 licence and the author attribution.

Allowed standard prerequisites (editorial).

- Primitive recursion schema. A function defined from primitive recursive g and h by f(0)=g, f(n+1)=h(n,f(n)) is primitive recursive.. It is the schema the displayed equations instantiate (738-747).
- Concatenation of Goedel numbers. The concatenation operation on coded symbol sequences is primitive recursive, established earlier in the unit, and Goedel numbers of individual symbols are constants.. It makes the recursion step a primitive recursive term in n and num(n) (738-747).

Proposition 0.1. *The function* $\mathrm{fn}\, num(n) = \mathrm{gn}\, \underline{n}$ *is primitive recursive.*

Proof. We define $\mathrm{fn}\, num(n)$ by primitive recursion:

$$\begin{aligned} \mathrm{fn}\, num(0) &= \mathrm{gn}\, 0 \\ \mathrm{fn}\, num(n+1) &= \mathrm{gn}\, \prime(\frown \mathrm{fn}\, num(n) \frown \mathrm{gn}\,). \end{aligned}$$

□

Proof. We define $\mathrm{fn}\, num(n)$ by primitive recursion:

$$\begin{aligned} \mathrm{fn}\, num(0) &= \mathrm{gn}\, 0 \\ \mathrm{fn}\, num(n+1) &= \mathrm{gn}\, \prime(\frown \mathrm{fn}\, num(n) \frown \mathrm{gn}\,). \end{aligned}$$

□